# 6CM521
# Frontiers in Artificial Intelligence and Data Science




By 100723791

Part B

Topic: Airport Security

Contents

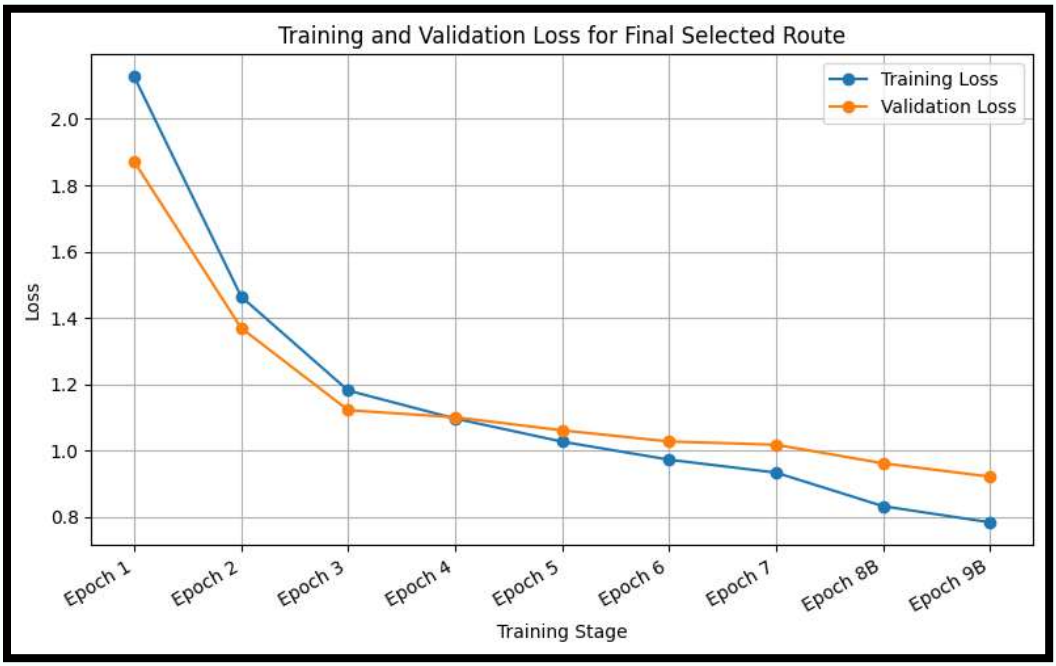
**1. Introduction** ..... 3
**2. Selection of a Model** ..... 3
**3. Dataset** ..... 4
**4. Implementation** ..... 4
**5. Results** ..... 4
**6. Integration** ..... 5
**7. Ethics and Safety** ..... 5
**8. Conclusion** ..... 6
**References** ..... 6

## 1. Introduction

In this report Part B is going to be analyzed. Part B is the selection of a model to tackle the training of the chosen database OPIXray. The database includes 8885 images and five classes [1]. The final model was YOLOv11 trained using Ultralytics framework. The prohibited item detection system uses an object detection model for identifying every class of item. All steps are recorded in the notebook and ReadMe files.

## 2. Selection of a Model

In Part A DETR deformable [3] was selected as the model that is going to be used for the training on the dataset. Main reasoning for that choice was its theoretical advantages into handling small objects. After running 9 epochs for the training with checkpoints and retreating into them (from 9 to 7) with two learning rates (firstly, 1e-5 and then 5e-6) the best checkpoint was epoch 9 (EPOCH9B_LR5E6) managed to reach 71.40% mAP@50 and 30.11% mAP@50:95. Every epoch after it, was leading to overfitting. The rise of the validation loss marks it. The Straight Knife class, for instance, reached only 11.97% mAP@50:95. Despite the extensive hyperparameter tuning, Deformable DETR kept showing limitations.




Thus, with these limitations YOLOv11 from the YOLO family was adopted as the final model [2]. As the latest stable release from 2024 it is more advanced and stable than YOLOv7 [4] (the model evaluated in Part A). This change had as a result an improvement of 17.2 percent (88.6% mAP@50) and a faster inference (2.3 milliseconds per image) which is needed for real-time airport applications.

## 3. Dataset

The OPIXray Dataset [1] is divided into 7109 images and 1772 validation ones and 4 additional images were removed from the validation set so that they could be used for the demo (essentially making them new to the model). It was split into five classes where there was imbalance revealed by Exploratory Data Analysis EDA (Part A), where the Straight knife class had only 809 instances, a huge gap from the 1635 Utility Knife instances. This imbalance in the data led to making the decision for enabling augmentation while training.

## 4. Implementation

Firstly, OPIXray annotations, originally in absolute coordinates (each line contains filename, class name, x_min, y_min, x_max, y_max) using a python script converted to the YOLO format of class_id, cx, cy, w, h all normalized to 0-1 relative to image dimensions. The YOLOv11n model was initialized with COCO pretrained weights and was fine-tuned for 50 epochs:

| **Parameter** | **Value** |
|---|---|
| **Epochs**<br>**Batch**<br>**Image size** | **50**<br>**16**<br>**640x640** |
| **Early stopping patience** | **10 epochs** |
| **Augmentation** | **Mosaic, flips, scaling** |
| **Training time** | **1.286 hours** |

Generalization was addressed through augmentation, early stopping and saving the best checkpoint.

## 5. Results

The model was evaluated in 1772 held out validation images and managed to have results as good as the following:

| Classes | Recall | mAP@50 | Precision | mAP@50-95 |
|---|---|---|---|---|
| Folding Knife | 0.923 | 0.932 | 0.887 | 0.431 |
| Straight Knife | 0.591 | 0.737 | 0.841 | 0.309 |
| Scissor | 0.970 | 0.976 | 0.977 | 0.510 |
| Utility Knife | 0.825 | 0.844 | 0.797 | 0.422 |
| Multi-tool Knife | 0.885 | 0.942 | 0.903 | 0.427 |

The classes that had the highest scores are the Scissor and Multi-tool knife classes because of their easily recognizable shapes [1]. The Straight Knife class had the most vulnerable shape and amount of instances thus having a recall of 0.591 which is weak. This aligns with the existing literature where thin objects can be confused easily.

## 6. Integration

The system is deployed as a REST API built with Flask [5]. It has three main endpoints which are GET /health (shows the service status), GET /info (shows metrics) and POST /detect (image input and returns JSON detection results with bounding boxes).

Comprehensive error handling is also added for http status codes 400, 404 and 500 responses. A Web User Interface (UI) is available at GET/ and holds up the demo. The API does not require special libraries and allows for easy integration with http requests.

## 7. Ethics and Safety

False negatives especially missed threats can present a serious risk when it comes to security. As specified in Part A the human factor should not be replaced and when Straight Knifes have a recall of 0.591 it is furthered confirmed. False positives are also a large, mostly unnecessary inconvenience and could cause disruption to the passengers. This can be caused by bias from the dataset, since the system was trained with certain data. Also, the x-rays must comply with the data protection rules and regulations.

## 8. Conclusion

The Key limitations are the Straight Knife recall remains low (0.591); mAP50-95 (0.420) which shows weaker localization under strict IoU thresholds and cross-dataset generalization was not evaluated. Future work should apply class-weighted loss for Straight Knife, test larger model variants (YOLOv11s/m), and evaluate on additional datasets.

The final system achieved 88.6% mAP@50 on OPIXray [1][2], a 17.2 point improvement over the initial Deformable DETR baseline [3]. The evidence-based model switch from DETR to YOLOv11 [2] demonstrates critical evaluation of approaches. The system is fully integrated as a REST API with a web demonstration interface, meeting all Part B requirements.

## References


[1] Wei, Y. et al. (2020). Occluded Prohibited Items Detection: An X-Ray Security Inspection Benchmark and De-occlusion Attention Module. ACM MM 2020. https://github.com/OPIXray-author/OPIXray

[2] Wang, A. et al. (2024). YOLO11: Real-Time Object Detection and Image Segmentation. Ultralytics. https://github.com/ultralytics/ultralytics

[3] Zhu, X. et al. (2021). Deformable DETR: Deformable Transformers for End-to-End Object Detection. ICLR 2021. https://arxiv.org/abs/2010.04159

[4] Wang, C.-Y. et al. (2022). YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors. CVPR 2023. https://arxiv.org/abs/2207.02696

[5] Flask Web Development Framework. Pallets Projects. https://flask.palletsprojects.com